



Hewlett Packard
Enterprise

HPE Cray EX

Programming and Optimization

Lab 3 - PBS Pro

June 30, 2026

Lab Environment Details

Your instructor will provide you with the details on the lab environment for this class. Please complete the following table with the lab information provided and use it for all lab exercises in this course.

Remember this information for the remaining labs.

User Access Node IP Address	
Training user ID	
Training User password	
Assigned Compute node Xname	
Assigned Compute node Nid	



Step 1 ☐	Log in to the login provided by your instructor with the credentials provided. Note: These exercises use the Bourne Again Shell (bash)
Step 2 ☐	Check the version of mpiexec that is available and then check whether the cray-pals module is loaded. If necessary, load the cray-pals module and check the version of mpiexec again. <pre>~> which mpiexec ~> mpiexec --version ~> module list ~> module load cray-pals ~> which mpiexec ~> mpiexec --version</pre> Note - HPE recommends using the version of mpiexec, or aprun, provided by the cray-pals module.
Step 3 ☐	Check the status of the compute nodes and the queue(s) on this cluster. <pre>~> pbsnodes -aS ~> qstat -Q</pre> How many cpus do most of the compute nodes have? _____
Sample pbs_test.sh script: <pre>#!/bin/bash #PBS -l select=4:ncpus=64 #PBS -j oe #PBS -q workq #PBS -l walltime=00:02:00 module load cray-pals mpiexec -n 4 --ppn 2 bash -c 'echo " Hostname: " \$(hostname) " PALS_RANKID: " \$PALS_RANKID " PALS_LOCAL_RANKID: " \$PALS_LOCAL_RANKID " PALS_NODEID: " \$PALS_NODEID' sort</pre>	
Step 4 ☐	Create a directory for your work in this lab and create a short script to like the example shown above. Feel free to add additional environment variables to the script. Be sure to change the mode of your script so that it is executable. <pre>~> mkdir pbs-lab ~> cd pbs-lab ~/pbs-lab> vi pbs_test.sh ~/pbs-lab> chmod 755 pbs_test.sh</pre>



Once ready run your sample script with PBS Pro

```
~/pbs-lab> qsub ./pbs_test.sh
```

What was the job id of your job? _____

Review the output file from the job

```
~/pbs-lab> cat pbs_test.sh.oJOB_ID
```

Sample OpenMP and MPI test program:

```
#define _GNU_SOURCE

#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <sched.h>
#include <mpi.h>
#include <omp.h>

/*
 * Borrowed from util-linux-2.13-pre7/schedutils/taskset.c
 */

static char *cpuset_to_cstr(cpu_set_t *mask, char *str)
{
    char *ptr = str;
    int i, j, entry_made = 0;
    for (i = 0; i < CPU_SETSIZE; i++) {
        if (CPU_ISSET(i, mask)) {
            int run = 0;
            entry_made = 1;
            for (j = i + 1; j < CPU_SETSIZE; j++) {
                if (CPU_ISSET(j, mask)) run++;
                else break;
            }
            if (!run)
                sprintf(ptr, "%d,", i);
            else if (run == 1) {
                sprintf(ptr, "%d,%d,", i, i + 1);
                i++;
            } else {
                sprintf(ptr, "%d-%d,", i, i + run);
                i += run;
            }
            while (*ptr != 0) ptr++;
        }
    }
    ptr -= entry_made;
    *ptr = 0;
    return(str);
}

int main(int argc, char *argv[])
{
    int rank, thread, duration=0;
    cpu_set_t coremask;
    char clbuf[7 * CPU_SETSIZE], hnbuf[64];

    if (argc == 2)
        duration = atoi(argv[1]);
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    memset(clbuf, 0, sizeof(clbuf));
    memset(hnbuf, 0, sizeof(hnbuf));
```



<pre>(void)gethostname(hnbuf, sizeof(hnbuf)); #pragma omp parallel private(thread, coremask, clbuf) { thread = omp_get_thread_num(); (void)sched_getaffinity(0, sizeof(coremask), &coremask); cpuset_to_cstr(&coremask, clbuf); #pragma omp barrier printf("Hello from rank %d, thread %d, on %s. (core affinity = %s)\n", rank, thread, hnbuf, clbuf); sleep(duration); } MPI_Finalize(); return(0); }</pre>	
Step 5 ☐	Copy the hybrid MPI - OpenMP c program above to your lab directory. ~/pbs-lab> vi hellope.c
Step 6 ☐	Compile the program you copied in step 7. ~/pbs-lab> cc -fopenmp -o hellope hellope.c
Step 7 ☐	Run your program in a single rank on the login node with 4 openmp threads. ~/pbs-lab> export OMP_NUM_THREADS=4 ~/pbs-lab> ./hellope How many lines of output did your job produce? _____
Sample job script to run MPI test program: #!/bin/sh #PBS -N hellope #PBS -j oe cd \$PBS_O_WORKDIR module load cray-pals export OMP_NUM_THREADS=8 mpiexec ./hellope	
Step 8 ☐	Create a script like the example above to run hellope and then use pbs pro to simply run the hellope application in a single mpi rank ~/pbs-lab> vi hellope1.sh ~/pbs-lab> qsub hellope1.sh Create additional copies of the script to run the hello pe application on multiple nodes and or in multiple mpi ranks. This will require a new PBS directive near the top of the script and additional parameters on your mpiexec line. Sample PBS directive to select 2 nodes each with 128 cores: #PBS -l select=2:ncpus=128 Sample mpiexec command to run two mpi ranks with 8 cores per rank



```
mpiexec -n 2 --depth 8 --cpu-bind depth ./hellope
```

Feel free to create additional scripts to test different combinations of PBS directives and `mpiexec` parameters.

Sample python MPI hello world program :

```
from mpi4py import MPI
import socket
import os

comm = MPI.COMM_WORLD

rank = comm.Get_rank()
size = comm.Get_size()

hostname = socket.gethostname()
pid = os.getpid()

try:
    affinity = sorted(os.sched_getaffinity(0))
    affinity = ",".join(map(str, affinity))
except Exception:
    affinity = "unknown"

print(
    f"Rank={rank:4d} "
    f"Size={size:4d} "
    f"PID={pid:6d} "
    f"Host={hostname:20s} "
    f"Affinity={affinity}"
)
```

Sample job script to run python hello world:

```
#!/bin/sh
#PBS -N hello_py
#PBS -l select=2:ncpus=128
#PBS -j oe

cd $PBS_O_WORKDIR
module load cray-pals
module load cray-python

mpiexec -n 4 --ppn 2 python hello-mpi.py | sort
```



Step 9

□

Using the samples above, try running a python version of the MPI hello world

```
~/pbs-lab> vi hello-mpi.py
~/pbs-lab> vi run_python_hello.sh
~/pbs-lab> module load cray-python
~/pbs-lab> qsub run_python_hello.sh
```

